Mnist - Dataset

Göran Olsson

2024-05-09

Mnist dataset användnig

- Träning och test av klassificeringsmodeller.
- Introduktion till neurala nätverk
- Optimering och utvärdering av prestationsförmåga.

- Mnist – bakgrund.
- Bilder utseende arkitektur.
- Prestanda. Vad göra?
- Skapa egna bilder. Appstöd.
- Bildhantering.
- Komplettera modellen med egna bilder. Hur?
- Träning av modell. (CPU/GPU?)
- Verifiering av modell. ”Rita siffror”
- Utvärdering av resultat.
- Test med olika modeller och optimerare.
- Produktionssättning av modell. Web applikation

Python-bibliotek för MNIST (alfabetisk ordning)

Bibliotek	Vanlig användning
Keras	Neural networks och CNN
Matplotlib	Visualisering av siffror och grafer
NumPy	Arrayer och numeriska beräkningar
OpenCV (`cv2`)	Bildbehandling
Pandas	Datahantering
Pillow (`PIL`)	Bildhantering
PyTorch	Deep learning och CNN
scikit-learn	Klassiska ML-modeller
Seaborn	Statistikgrafer och heatmaps
Streamlit	Webbappar för MNIST
TensorFlow	Deep learning och CNN

Vad kan man göra för att förbättra prestationsförmågan?

- Mer data.
- EDA - databehandling
- Bättre kvalitet på datasetet.
- Bättre Modell
- Optimering av modellen.

[7]:
```
(x_train, y_train), (x_test, y_test) = mnist.load_data()


print(x_train.shape)
print(y_train.shape)

print(x_test.shape)
print(y_test.shape)

print(type(x_train))
print(type(y_train))
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
```

Bild egenskaper

* 28x28 pixlar
* Gråskala
* Normaliserad (0-1)

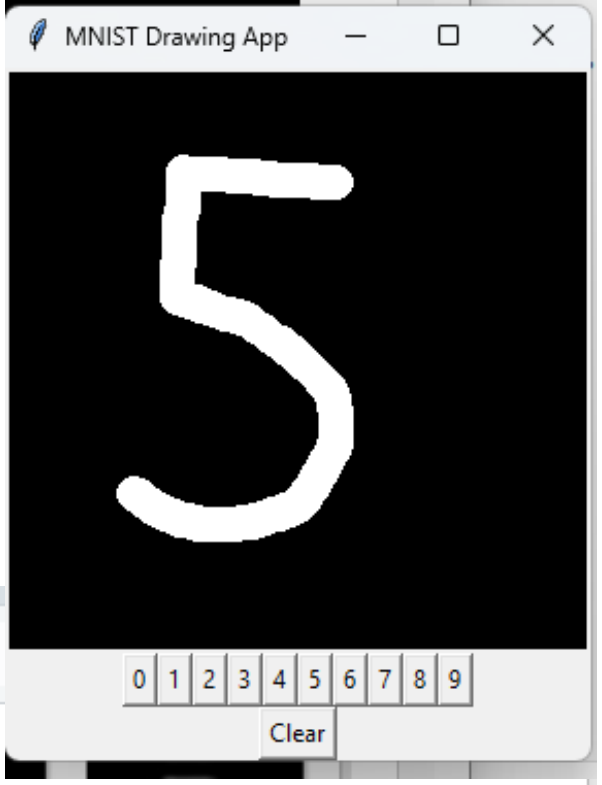
```
img = img.resize((28,28)).convert("L")
img = np.array(img) / 255.0
img = 1 - img   # ofta behövs invertering
```

AI Kurs Yrkeshögskolan > AI Kurs 2026 > custom_digits >

Sök i

Sortera Visa

Namn	Senast ändrad	Typ	Storl
0	2026-04-02 19:17	Filmapp	
1	2026-04-02 19:21	Filmapp	
2	2026-04-02 19:23	Filmapp	
3	2026-04-02 19:24	Filmapp	
4	2026-04-02 19:25	Filmapp	
5	2026-04-02 19:26	Filmapp	
6	2026-04-02 19:28	Filmapp	
7			
8			
9			
MNIST			



AI Kurs Yrkeshögskolan > AI Kurs 2026 > custom_digits > 5

Sortera Visa

Typisk preprocessing-kedja

Python

```python
img = Image.open("digit.png")

img = img.convert("L")

img = img.resize((28,28))

img_array = np.array(img)

img_array = img_array / 255.0

img_flat = img_array.reshape(1,784)
```

Viktigaste MNIST-formatet

Egenskap	Värde
Storlek	28×28
Grayscale	Ja
Bakgrund	svart
Siffra	vit
Normaliserad	0–1

Python-kommandon för att anpassa egna bilder till MNIST

(alfabetisk ordning efter funktion/bibliotek)

Kommando	Användning
`Image.fromarray()`	skapa PIL-bild från numpy-array
`Image.open()`	öppna bildfil
`img.resize((28,28))`	ändra storlek till MNIST-format
`img.convert("L")`	konvertera till grayscale
`np.array()`	PIL → numpy-array
`np.count_nonzero()`	kontrollera om något ritats
`np.expand_dims()`	lägga till dimension för CNN
`np.invert()`	invertera svart/vitt
`reshape(1,784)`	flatten för sklearn-modeller
`reshape(1,28,28,1)`	input-format för CNN
`StandardScaler()`	normalisering/scaling
`X / 255.0`	normalisering till 0–1

Modell	Typisk Accuracy	Kommentar
CNN	99%+	Bäst för bilder och handritade siffror
KNeighborsClassifier (KNN)	97%	Enkel men långsam prediction
LinearSVC	94–96%	Snabb och bra
LogisticRegression	92–94%	Enkel klassisk modell
MLPClassifier	96–98%	Neural network i sklearn
PassiveAggressiveClassifier	90–93%	Extremt snabb
Perceptron	85–90%	Enkel neural modell
RandomForestClassifier	95–97%	Robust och lätt att använda
SGDClassifier	88–93%	Mycket snabb
SVC (RBF kernel)	97–98%	Hög accuracy men långsam träning

Kort rekommendation

Om du vill ha	Modell
Bäst accuracy	CNN
Snabbast träning	SGDClassifier
Enkel och bra	MLPClassifier
Klassisk ML	SVC
Snabb webbapp	LinearSVC
Enklast för nybörjare	LogisticRegression

MLP eller CNN

- MLP (Multilayer Perceptron)

Fördelar:

- Enkelt att förstå och implementera
- Fungerar bra på strukturerad data (tabeller, features)
- Snabbare att träna på små dataset

Nackdelar:

- Tar inte hänsyn till rumslig struktur (t.ex. bilder)
- Många parametrar → risk för överanpassning
- Sämre prestanda på bilddata

- CNN (Convolutional Neural Network)

Fördelar:

- Mycket bra på bild- och videoanalys
- Färre parametrar än MLP för bilder
- Kan automatiskt hitta viktiga features

Nackdelar:

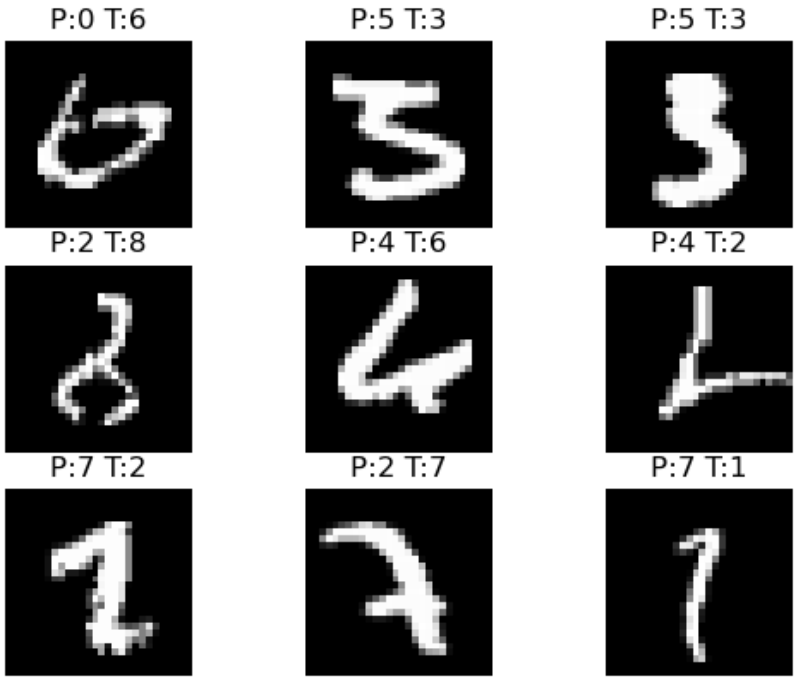
- Mer komplext att bygga och förstå
- Kräver mer beräkningskraft
- Långsammare att träna

⚖️ Enkel tumregel

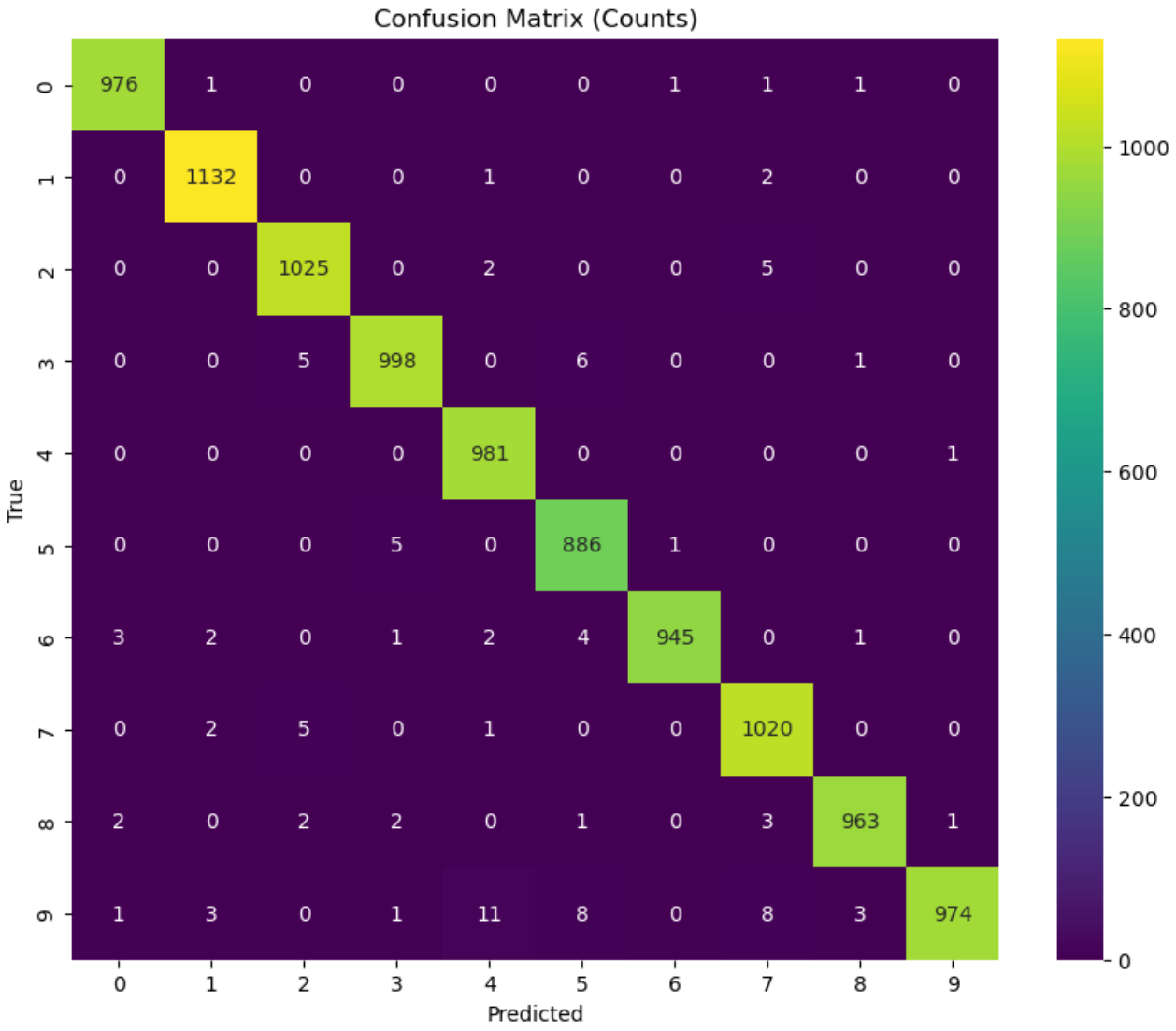
Problemtyp	Bäst modell
Tabell / siffror	✅ MLP
Bilder	✅ CNN
Sekvenser/text	RNN / Transformer
Liten data	Ofta MLP

Number of wrong predictions: 100

```
[25]: import matplotlib.pyplot as plt

for i in range(9):
    idx = wrong[i]

    plt.subplot(3, 3, i+1)
    plt.imshow(x_test[idx], cmap='gray')
    plt.title(f"P:{pred_labels[idx]} T:{y_test[idx]}")
    plt.axis('off')

plt.show()
```

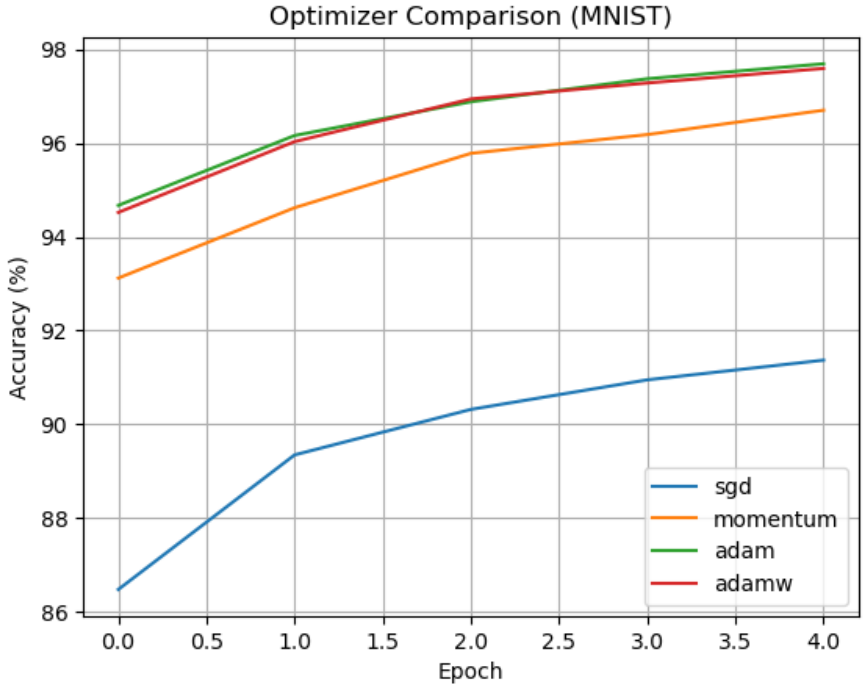


Confusion Matrix - CNN



optimizers = ["sgd", "momentum", "adam", "adamw"]

```
Training with sgd
sgd - Epoch 1: 86.48%
sgd - Epoch 2: 89.35%
sgd - Epoch 3: 90.32%
sgd - Epoch 4: 90.95%
sgd - Epoch 5: 91.37%

Training with momentum
momentum - Epoch 1: 93.12%
momentum - Epoch 2: 94.62%
momentum - Epoch 3: 95.78%
momentum - Epoch 4: 96.18%
momentum - Epoch 5: 96.70%

Training with adam
adam - Epoch 1: 94.67%
adam - Epoch 2: 96.16%
adam - Epoch 3: 96.88%
adam - Epoch 4: 97.37%
adam - Epoch 5: 97.69%

Training with adamw
adamw - Epoch 1: 94.52%
adamw - Epoch 2: 96.03%
adamw - Epoch 3: 96.94%
adamw - Epoch 4: 97.28%
adamw - Epoch 5: 97.59%
```



MNIST MLP och CNN jämförelse

Rita en siffra (0–9)

MLP accuracy: 0.9759

CNN accuracy: 0.9866

Skicka

MLP: 4 (61.8%)

CNN: 1 (85.2%)

28x28-bild som modellerna ser

MLP sannolikheter

0: 0.31%

1: 26.13%

2: 0.09%

MNIST MLP och CNN jämförelse

Rita en siffra (0–9)

MLP accuracy: 0.9759

CNN accuracy: 0.9866

Skicka

MLP: 7 (84.6%)

CNN: 8 (30.0%)

28x28-bild som modellerna ser

MLP sannolikheter

0: 0.22%

1: 0.00%

2: 0.14%

MNIST MLP och CNN jämförelse

Rita en siffra (0–9)

MLP accuracy: 0.9759

CNN accuracy: 0.9866

Skicka

MLP: 4 (81.3%)

CNN: 4 (78.5%)

28x28-bild som modellerna ser

MLP sannolikheter

0: 0.00%

1: 0.05%

2: 0.06%

3: 5.65%

4: 81.32%